Le watchdog du PIC16F84A

Polo Antoine

1. INTRODUCTION

Le watchdog permet de réinitialiser le microcontrôleur si aucun appel à `CLRWDT()` n'a été fait dans le temps imparti. C'est une sécurité : si le programme se bloque (boucle infinie, plantage…), le watchdog n'est plus remis à zéro et le PIC redémarre de lui-même.

Il s'active avec le bit de configuration `WDTE` (`#pragma config WDTE = ON`). Dans ce document, nous allons voir comment régler sa durée et comment savoir, au démarrage, si le PIC vient d'être réinitialisé par le watchdog.

2. REGISTRES IMPORTANTS

Les deux registres importants sont `OPTION_REG` et `STATUS`.

2.1. Registre de configuration (OPTION_REG)

Ce registre de 8 bits configure le prédiviseur, qui peut être assigné au Timer0 ou au watchdog, ainsi que quelques fonctions du PORTB.

Table 1. Bits du registre OPTION_REG.

b7	b6	b5	b4	b3	b2	b1	b0
nRBPU	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0

- `nRBPU` : contrôle les résistances de pull-up du PORTB ; le pull-up est activé à 0.
- `INTEDG` : sens de l'interruption externe (RB0), 1 pour front montant et 0 pour descendant.
- `T0CS` : source d'horloge du timer, 0 pour l'horloge interne et 1 pour l'horloge externe.
- `T0SE` : front d'horloge externe (si T0CS à 1), 0 pour front montant et 1 pour descendant.
- `PSA` : assignation du prédiviseur, à 0 il est assigné à Timer0 et à 1 pour le watchdog.
- `PS2-0` : détermine la valeur du prédiviseur. Il utilise une valeur binaire de 000 à 111.

Pour le watchdog, il faut donc mettre `PSA` à 1. Les bits `PS2-0` correspondent alors aux valeurs de la Table 2, qui diffèrent de celles du Timer0 : le prédiviseur du watchdog va de 1 à 128.

Table 2. Durée du watchdog selon PS2-0 (PSA = 1).

PS2-0	Division	Durée
000	1:1	18 ms
001	1:2	36 ms
010	1:4	72 ms
011	1:8	144 ms
100	1:16	288 ms
101	1:32	576 ms
110	1:64	1152 ms
111	1:128	2304 ms

2.2. Registre d'état (STATUS)

Table 3. Bits du registre STATUS.

b7	b6	b5	b4	b3	b2	b1	b0
IRP	RP1	RP0	nTO	nPD	Z	DC	C

- `IRP` : non utilisé, pour les PIC ayant plus de mémoire.
- `RP1` : non utilisé, pour des plus gros PIC.
- `RP0` : sélectionne la banque mémoire, 0 pour la banque 0 et 1 pour la banque 1.
- `nTO` : indique si le démarrage est normal (1) ou un reset par le watchdog (0).
- `nPD` : passe à 0 si le PIC a été mis en veille via un `sleep`, passe à 1 au démarrage.
- `Z` : c'est le *Zero Flag*, passe à 1 si la dernière opération mathématique ou logique a donné un résultat de 0.
- `DC` : indique un débordement du 4e bit vers le 5e lors d'une addition.
- `C` : *Carry*, indique un débordement général, la valeur est supérieure à 255.

Pour le watchdog, c'est surtout le bit `nTO` qui nous intéresse : il permet au programme de savoir pourquoi le PIC vient de démarrer.

3. DURÉE DU WATCHDOG

Pour déterminer le temps maximum entre chaque appel à `CLRWDT()`, il suffit de prendre le temps moyen du watchdog, qui est de 18 ms, et de le multiplier par le prédiviseur. Cette valeur varie en fonction de la tension d'alimentation et de la température.

Le minimum est donc de 18 ms et le maximum est de $18 \times 128 = 2304$ ms (soit 2,3 s). C'est le mode le plus utilisé.

4. EXEMPLE DE CODE

4.1. Montage

Le montage reprend le circuit de base de l'introduction au PIC16F84A (quartz de 8 MHz, condensateurs de 22 pF et résistance de 10 kΩ sur MCLR), auquel on ajoute :

- trois LEDs, chacune avec sa résistance de limitation, sur les broches RB1, RB2 et RB3 ;
- un bouton poussoir entre la broche RB0 et la masse, sans résistance externe puisque le pull-up interne du PORTB est activé.

4.2. Code

Le programme utilise 3 LEDs et un bouton poussoir. Une première LED indique si le démarrage est normal ou s'il fait suite à un reset du watchdog, et la LED sur RB3 clignote et reste allumée quand on appuie sur le bouton RB0.

```
#pragma config FOSC = HS
#pragma config WDTE = ON
#pragma config PWRTE = OFF
#pragma config CP = OFF

#include <xc.h>

#define _XTAL_FREQ 8000000

void main(void) {
    TRISB = 0b00000001;
    PORTB = 0x00;

    OPTION_REG = 0b00001111;

    if (STATUSbits.nTO) {
        PORTBbits.RB1 = 1;
    } else {
        PORTBbits.RB2 = 1;
    }

    __delay_ms(1000);
```

```
    PORTBbits.RB1 = 0;
    PORTBbits.RB2 = 0;

    while (1) {
        PORTBbits.RB3 = 1;
        __delay_ms(250);
        PORTBbits.RB3 = 0;
        __delay_ms(250);

        CLRWDT();

        if (PORTBbits.RB0 == 0) {
            PORTBbits.RB3 = 1;
            while (1);
        }
    }
}
```

Code 1. Détection d'un reset par le watchdog.

4.3. Fonctionnement

- **Configuration** : `TRISB = 0b00000001` configure RB0 en entrée (le bouton) et les autres broches du PORTB en sorties, puis `PORTB = 0x00` éteint toutes les LEDs. `OPTION_REG = 0b00001111` donne `nRBPU = 0` (pull-up activés), `PSA = 1` (prédiviseur assigné au watchdog) et `PS2-0 = 111`, soit une division par 128 et une durée d'environ 2,3 s.
- **Cause du démarrage** : `STATUSbits.nTO` vaut 1 lors d'un démarrage normal, la LED de RB1 s'allume alors. Après un reset par le watchdog, il vaut 0 et c'est la LED de RB2 qui s'allume. Après une seconde (`__delay_ms(1000)`), les deux LEDs sont éteintes.
- **Boucle principale** : la LED de RB3 clignote avec deux délais de 250 ms, puis `CLRWDT()` remet le watchdog à zéro. Il est appelé toutes les 500 ms, bien avant les 2,3 s du watchdog.
- **Appui sur le bouton** : RB0 passe à 0, la LED de RB3 est allumée et le programme entre dans une boucle infinie sans appeler `CLRWDT()`. Au bout d'environ 2,3 s, le watchdog réinitialise le PIC. Le programme redémarre avec `nTO = 0`, ce qui allume la LED de RB2, puis le clignotement reprend.

5. CONCLUSION

Le watchdog est une sécurité simple à mettre en place : on l'active avec `WDTE`, on règle sa durée avec le prédiviseur de `OPTION_REG`, et le programme doit appeler `CLRWDT()` régulièrement tant qu'il fonctionne normalement. Le bit `nTO` du registre `STATUS` permet ensuite de savoir au démarrage si le PIC a été réinitialisé par le watchdog.